Python Quick Revision Notes

SRE Role | Java Background | Networking Domain

Chapter 1 — Modules, pip & REPL

1.1 Modules

```
import os
from os import path
import os as op_sys
```

SRE Modules

- **os:** filesystem,env
- **sys:** args,exit
- **subprocess:** shell cmds
- **json:** API parse
- **re:** regex/logs
- **socket:** network
- **time:** sleep,timestamps

1.2 pip

```
pip3 install requests
pip3 freeze > requirements.txt
python3 -m venv myenv  &&  source myenv/bin/activate
```

> *Always use pip3. One venv per project.*

Chapter 2 — Variables, Typecasting & User Input

2.1 Variables

No type declaration. snake_case. Dynamic typing.

Java	Python

`int age=25;`
`String n="web01";`
`boolean b=true;`
	`age=25`
	`n='web01'`
	`b=True`

2.2 Data Types

Type	Example	Note
`str`	`'web01'`	text
`int`	`8080`	whole

float	99.5	decimal
bool	True	capital T/F
None	None	null

2.3 Typecasting

```
int('8080')  float('3.14')  str(404)  bool(0)
```

> □ *input() always returns str — cast on same line: port=int(input('Port: '))*

Chapter 3 — Strings & Escape Characters

3.1 Strings

```
s='web01'  len(s)  s[0]  s[-1]  s[1:4]  s[::-1]
```

3.2 Escape Chars

Escape	Meaning
\n	newline
\t	tab
\\	backslash
r'...'	raw string

3.3 Key Methods

```
s.strip() s.split(',') ','.join(lst) s.replace(a,b)
s.upper() s.lower() s.startswith() s.endswith()
s.find('x')  'x' in s  s.isdigit()
f"Host {host} port {port}"
```

Chapter 4 — Lists & Tuples

4.1 Lists

```
h=['web01','web02']  h.append(x)  h.remove(x)  h.pop(i)
h.sort()  sorted(h)  h.index(x)  x in h  len(h)
[x*2 for x in h if x.startswith('web')]  # comprehension
```

> □ *sort() modifies original. sorted() returns new list.*

4.2 Tuples

```
t=('web01',8080)  t[0]  host,port=t  a,b=b,a
single=(42,)  # trailing comma! t.count(x) t.index(x)
```

	List [ ]	Tuple ( )
Mutable	Yes	No
Use	data changes	data fixed

Chapter 5 — Dictionaries & Sets

5.1 Dicts

```
d={'host':'web01','port':8080}
d.get('key',default)  d.keys()  d.values()  d.items()
for k,v in d.items():  d.update({k:v})  d.pop(k)
{k:v for k,v in d.items() if v>50}  # dict comprehension
```

> □ *Always use .get(key,default) to avoid KeyError.*

5.2 Sets

```
s={1,2,3}  empty=set()  s.add(x)  s.discard(x)
A|B  A&B  A-B  A^B  list(set(lst))  # dedup
```

	List	Tuple	Dict	Set
Mutable	Yes	No	Yes	Yes
Ordered	Yes	Yes	Yes	No
Dups	Yes	Yes	No	No

Chapter 6 — Conditional Expressions

6.1 if/elif/else

```
if cpu>90:
    print('CRIT')
elif cpu>70:
    print('WARN')
else:
    print('OK')
```

6.2 Operators

```
== != > < >= <=  in not in  is is not
and  or  not
```

6.3 Truthy/Falsy

```
Falsy: 0 '' [] {} None False
if hosts:  # True if not empty   if not errors:  # True if empty
```

6.4 Ternary

```
status='UP' if active else 'DOWN'
print(f"{'UP' if active else 'DOWN'}")
```

6.5 match-case (3.10+)

```
match code:
    case 200: ok()
    case 301|302: redirect()
    case _: unknown()
```

Chapter 7 — Loops

7.1 for Loop

```
for host in hosts:
for i in range(5):        # 0..4
for i in range(1,6):      # 1..5
for i in range(10,0,-1):  # countdown
```

7.2 while Loop

```
while count < 3:
    count += 1
while True:
    if done: break
```

7.3 break/continue/pass

Keyword	Does what
break	exit loop now
continue	skip this iteration
pass	placeholder — do nothing

7.4 enumerate & zip

```
for i, val in enumerate(lst, 1):   # idx from 1
for a, b in zip(list1, list2):     # parallel loop
```

7.5 for/else

```
for x in lst:
    if bad: break
else:
    print('no break hit')  # unique to Python
```

Chapter 8 — Functions & Recursion

def, args, *args, **kwargs, return, lambda, recursion

8.1 Defining & Calling Functions

A function is a reusable block of code. Define once, call many times. Like Java methods.

Java	Python

`public int add(int a, int b){`
`  return a + b;`
`}`
`add(3, 4);`
	`def add(a, b):`
	`  return a + b`
	
	`add(3, 4)`

```
# Basic function
def greet(name):
    print(f'Hello {name}')

greet('Rahul')    # Hello Rahul

# Function with return
def add(a, b):
    return a + b

result = add(3, 4)
print(result)     # 7

# Function with no return → returns None
def show(msg):
    print(msg)    # no return

x = show('hi')   # x = None
```

Python functions return None by default if no return statement. Like void in Java.

8.2 Default Arguments

Give parameters a default value — caller can skip them.

```
def connect(host, port=22, timeout=5):
    print(f'Connecting to {host}:{port} timeout={timeout}s')

connect('web01')              # uses port=22, timeout=5
connect('web01', 8080)        # port=8080, timeout=5
connect('web01', 8080, 10)    # all custom

# Keyword arguments — any order
connect('web01', timeout=30, port=443)
```

Default mutable argument bug — NEVER use list/dict as default: def f(x, lst=[]). Use None instead.

```
# WRONG — shared across all calls!
def add_host(host, lst=[]):
    lst.append(host)
    return lst

# CORRECT
def add_host(host, lst=None):
    if lst is None:
        lst = []
    lst.append(host)
    return lst
```

8.3 *args — Variable Positional Arguments

Accept any number of positional arguments. Stored as a tuple inside function.

Java	Python

`// Java varargs`
`public void log(String...msgs){`
`  for(String m:msgs)`
`  print(m);`
`}`
	`def log(*msgs):`
	`  for m in msgs:`
	`  print(m)`

```
def log(*msgs):
    for m in msgs:
        print(m)

log('Server up')                    # 1 arg
log('CPU high','MEM high','OK')     # 3 args

# *args is a TUPLE inside function
def total(*nums):
    print(type(nums))    # <class 'tuple'>
    return sum(nums)

total(10, 20, 30, 40)    # 100

# Mix normal + *args
def alert(level, *hosts):
    for h in hosts:
        print(f'[{level}] {h} is DOWN')

alert('CRITICAL', 'web01', 'web02', 'db01')
```

> **args collects extra positional args as a tuple. Name 'args' is convention — could be *anything.*

8.4 **kwargs — Variable Keyword Arguments

Accept any number of keyword arguments. Stored as a dict inside function.

```
def server_info(**kwargs):
    for k, v in kwargs.items():
        print(f'{k}: {v}')

server_info(host='web01', port=8080, region='us-east')
# host: web01
# port: 8080
# region: us-east

# type of kwargs inside function = dict
def show(**data):
    print(type(data))   # <class 'dict'>

# Mix all argument types
def full(name, *args, **kwargs):
    print(name)     # normal arg
    print(args)     # tuple of extra positional
    print(kwargs)   # dict of keyword args

full('web01', 80, 443, region='us', active=True)
# web01
# (80, 443)
# {'region': 'us', 'active': True}
```

> *Order must be: normal args → *args → **kwargs. Never swap this order.*

8.5 return — Returning Values

```
# Return single value
def square(n):
    return n * n

# Return multiple values (as tuple)
def min_max(lst):
    return min(lst), max(lst)   # returns tuple

lo, hi = min_max([3,1,9,4])    # unpack
print(lo, hi)   # 1  9

# Early return
def divide(a, b):
    if b == 0:
        return None    # early exit
    return a / b

# Return dict — very common in SRE scripts
def check_host(host):
    return {
        'host'  : host,
        'status': 'UP',
        'cpu'   : 45.2
    }

result = check_host('web01')
print(result['status'])   # UP
```

Python functions can return multiple values — actually returns a tuple. Unpack with a,b = func()

8.6 Scope — Local vs Global

Variable scope works like Java — local variables live inside the function only.

```
x = 10   # global

def show():
    x = 99   # local — different from global x
    print(x) # 99

show()
print(x)     # 10 — global unchanged

# To modify global inside function — use 'global'
count = 0

def increment():
    global count
    count += 1

increment()
print(count)  # 1
```

Avoid using 'global' — makes code hard to debug. Pass values as arguments instead.

8.7 lambda — Anonymous Functions

A one-line function with no name. Like Java lambda expressions.

Java	Python

`// Java lambda`
`Function<Integer,Integer> sq`
`  = x -> x * x;`
	`# Python lambda`
	`sq = lambda x: x * x`
	`sq(5)  # 25`

```
# Basic lambda
square = lambda x: x * x
print(square(5))     # 25

add = lambda a, b: a + b
print(add(3, 4))     # 7

# Most common use — with sorted(), map(), filter()

# Sort list of dicts by a key
servers = [
    {'host':'web01','cpu':90},
    {'host':'db01', 'cpu':45},
    {'host':'web02','cpu':70},
]
sorted_by_cpu = sorted(servers, key=lambda s: s['cpu'])
# [db01:45, web02:70, web01:90]

# filter — keep only DOWN servers
hosts = [{'name':'web01','up':True},{'name':'db01','up':False}]
down  = list(filter(lambda h: not h['up'], hosts))

# map — transform every item
nums   = [1, 2, 3, 4]
doubled = list(map(lambda x: x*2, nums))  # [2,4,6,8]
```

lambda = quick one-liner function. Use def for anything more than one expression.

8.8 Recursion

A function that calls itself. Must have a base case to stop — else infinite loop / stack overflow.

Java	Python

`// Java recursion`
`int fact(int n){`
`  if(n==0) return 1;`
`  return n * fact(n-1);`
`}`
	`def factorial(n):`
	`  if n == 0:  # base case`
	`  return 1`
	`  return n * factorial(n-1)`

```
# Factorial
def factorial(n):
    if n == 0:          # BASE CASE — must have this!
        return 1
    return n * factorial(n - 1)

print(factorial(5))   # 120
# factorial(5) = 5 * factorial(4)
#              = 5 * 4 * factorial(3)
#              = 5 * 4 * 3 * 2 * 1 = 120

# Fibonacci
def fibonacci(n):
    if n <= 1:          # base case
```

```
        return n
    return fibonacci(n-1) + fibonacci(n-2)

print(fibonacci(6))   # 8  (0,1,1,2,3,5,8)

# Sum of list using recursion
def rec_sum(lst):
    if len(lst) == 0:   # base case
        return 0
    return lst[0] + rec_sum(lst[1:])

print(rec_sum([1,2,3,4,5]))  # 15
```

> *Always define a BASE CASE first. Without it, recursion runs forever → RecursionError.*

> *Python default recursion limit = 1000 calls. Check with: import sys; sys.getrecursionlimit()*

8.9 Chapter 8 — Quick Cheatsheet

Topic	Syntax / Example
Define function	`def greet(name): return f'Hi {name}'`
Call function	`greet('Rahul')`
Default arg	`def connect(host, port=22):`
Keyword arg	`connect('web01', timeout=30)`
*args (tuple)	`def log(*msgs): for m in msgs:`
**kwargs (dict)	`def info(**kw): for k,v in kw.items():`
Argument order	`def f(a, *args, **kwargs):`
Return single	`return value`
Return multiple	`return a, b → x,y = func()`
Local scope	variable inside def — not visible outside
Global keyword	`global x → modify global (avoid!)`
lambda	`sq = lambda x: x*x → sq(5)`
lambda + sorted	`sorted(lst, key=lambda x: x['cpu'])`
lambda + filter	`list(filter(lambda x: x>0, nums))`
lambda + map	`list(map(lambda x: x*2, nums))`
Recursion	`def f(n): if base: return ... return f(n-1)`
Recursion limit	`sys.getrecursionlimit() # default 1000`

8.10 Real SRE Function Example

```
import json

def check_servers(*hosts, threshold=80, **options):
    '''
    Check multiple servers for resource usage.
    threshold: alert if cpu/mem above this %
    options: extra config like region, team
    '''
    results = []

    for host in hosts:
```

```
        # Simulate check (replace with real API/SSH call)
        cpu = 85   # would come from monitoring API
        mem = 72

        status = 'CRITICAL' if cpu > threshold else 'OK'

        results.append({
            'host'   : host,
            'cpu'    : cpu,
            'mem'    : mem,
            'status' : status,
            'region' : options.get('region', 'unknown')
        })

    return results

data = check_servers('web01','web02','db01', threshold=80, region='us-east')
for d in data:
    print(f"[{d['status']}] {d['host']} CPU:{d['cpu']}%")
```

Interview Questions & Answers — All Chapters

Section A — Core Python Concepts

Q1. What is the difference between mutable and immutable?

```
# Mutable   — can change after creation
lst = [1,2,3]
lst.append(4)       # OK — list is mutable

# Immutable — cannot change after creation
s = 'hello'
s[0] = 'H'          # ✗ TypeError — str is immutable

# Mutable:   list, dict, set
# Immutable: int, float, str, tuple, bool, None
```

Q2. What is the difference between 'is' and '=='?

```
a = [1,2,3]
b = [1,2,3]
a == b    # True  — same VALUE
a is b    # False — different objects in memory

# 'is' is like comparing object references in Java
# Use 'is' ONLY for: None, True, False
if x is None:
if x is not None:
```

Q3. What is *args and **kwargs?

```
# *args   — variable positional args → stored as TUPLE
def f(*args):   print(type(args))  # <class 'tuple'>

# **kwargs — variable keyword args  → stored as DICT
def f(**kwargs): print(type(kwargs))  # <class 'dict'>

# Order rule: def f(normal, *args, **kwargs)
```

Q4. What is list comprehension? When to use?

```
# Instead of:
result = []
for x in range(10):
    if x % 2 == 0:
        result.append(x*2)

# Use comprehension:
result = [x*2 for x in range(10) if x%2==0]
# [0,4,8,12,16]

# Use when: simple filter + transform in one line
# Avoid when: complex logic — use for loop instead
```

Q5. Difference between sort() and sorted()?

```
lst = [3,1,4,1,5]

lst.sort()        # modifies ORIGINAL, returns None
new = sorted(lst) # returns NEW list, original unchanged

# With lambda
sorted(servers, key=lambda s: s['cpu'], reverse=True)
```

Q6. What is a lambda function?

```
# Anonymous one-line function
sq = lambda x: x*x     # same as: def sq(x): return x*x

# Common use cases
sorted(lst, key=lambda x: x['name'])
list(filter(lambda x: x>0, nums))
list(map(lambda x: x*2, nums))
```

Q7. What is recursion? What is a base case?

```
# Recursion = function calls itself
# Base case = condition to STOP recursion

def factorial(n):
    if n == 0: return 1   # BASE CASE — must have!
    return n * factorial(n-1)

# Without base case → RecursionError (stack overflow)
```

Q8. What is the difference between append() and extend()?

```
lst = [1,2,3]

lst.append([4,5])   # adds list AS ONE item → [1,2,3,[4,5]]
lst.extend([4,5])   # adds each item        → [1,2,3,4,5]

# append = add one item   extend = merge another list
```

Q9. How does Python handle memory — shallow vs deep copy?

```
import copy

original = [[1,2],[3,4]]
```

```
# Shallow copy — copies outer, inner still shared
shallow = original.copy()       # or copy.copy(original)
shallow[0].append(99)
print(original)  # [[1,2,99],[3,4]]  ← original changed!

# Deep copy — fully independent
deep = copy.deepcopy(original)
deep[0].append(99)
print(original)  # [[1,2],[3,4]]  ← original safe
```

Q10. What is GIL (Global Interpreter Lock)?

GIL is a mutex in CPython that allows only ONE thread to execute Python bytecode at a time. Even on multi-core CPUs, threads don't truly run in parallel for CPU-bound tasks. Use multiprocessing for true parallelism.

```
# GIL affects CPU-bound threads
# I/O-bound tasks (network, disk) still benefit from threading
# For CPU parallelism: use multiprocessing module
```

Section B — Function-Specific Questions

Q11. What happens if you use a mutable default argument?

```
# BUG — list is created ONCE and shared across calls
def add(item, lst=[]):
    lst.append(item)
    return lst

add('a')  # ['a']
add('b')  # ['a','b']  ← BUG! expected ['b']

# FIX — use None as default
def add(item, lst=None):
    if lst is None: lst = []
    lst.append(item)
    return lst
```

Q12. Can a Python function return multiple values?

```
def stats(lst):
    return min(lst), max(lst), sum(lst)/len(lst)

lo, hi, avg = stats([10,20,30,40])
print(lo, hi, avg)   # 10 40 25.0

# It actually returns a TUPLE — unpacking makes it look like multiple
```

Q13. What is the difference between local and global scope?

```
x = 'global'

def test():
    x = 'local'    # new local variable, global unchanged
    print(x)       # local

test()
print(x)   # global
```

```
# To modify global:
def modify():
    global x
    x = 'changed'
```

Section C — Tricky / Output Questions

Q14. What is the output?

```
def f(x, lst=[]):
    lst.append(x)
    return lst

print(f(1))   # [1]
print(f(2))   # [1, 2]   ← NOT [2]! Mutable default bug.
print(f(3))   # [1, 2, 3]
```

Q15. What is the output?

```
x = 5
def change():
    x = 10
    print('inside:', x)

change()         # inside: 10
print('outside:',x)  # outside: 5  ← global unchanged
```

Q16. What is the output?

```
def add(a, b=10):
    return a + b

print(add(5))       # 15  (b uses default 10)
print(add(5, 20))   # 25  (b=20)
print(add(b=3,a=2)) # 5   (keyword args, any order)
```

Q17. What is the output of this recursion?

```
def mystery(n):
    if n == 0:
        return 0
    return n + mystery(n-1)

print(mystery(4))
# mystery(4) = 4 + mystery(3)
#            = 4 + 3 + mystery(2)
#            = 4 + 3 + 2 + mystery(1)
#            = 4 + 3 + 2 + 1 + mystery(0)
#            = 4 + 3 + 2 + 1 + 0 = 10
```

Basic Programs — Commonly Asked in Interviews

Program 1 — Reverse a String

```
# Method 1: Slicing (best)
def reverse_str(s):
    return s[::-1]

# Method 2: loop
def reverse_str2(s):
    result = ''
    for ch in s:
        result = ch + result
    return result

print(reverse_str('hello'))   # olleh
print(reverse_str('web01'))   # 10bew
```

Program 2 — Check Palindrome

```
def is_palindrome(s):
    s = s.lower().replace(' ','')   # normalize
    return s == s[::-1]

print(is_palindrome('racecar'))  # True
print(is_palindrome('hello'))    # False
print(is_palindrome('A man a plan a canal Panama'))  # True
```

Program 3 — Factorial (loop + recursion)

```
# Loop version
def factorial_loop(n):
    result = 1
    for i in range(1, n+1):
        result *= i
    return result

# Recursive version
def factorial_rec(n):
    if n == 0: return 1
    return n * factorial_rec(n-1)

print(factorial_loop(5))   # 120
print(factorial_rec(5))    # 120
```

Program 4 — Fibonacci Series

```
# Print first n fibonacci numbers
def fibonacci(n):
    a, b = 0, 1
    for _ in range(n):
        print(a, end=' ')
        a, b = b, a + b

fibonacci(8)   # 0 1 1 2 3 5 8 13

# Recursive (slower)
def fib_rec(n):
    if n <= 1: return n
    return fib_rec(n-1) + fib_rec(n-2)
```

Program 5 — Check Prime Number

```
def is_prime(n):
    if n < 2: return False
    for i in range(2, int(n**0.5)+1):  # only check to sqrt(n)
        if n % i == 0:
            return False
    return True

print(is_prime(7))   # True
print(is_prime(10))  # False
print(is_prime(13))  # True
```

Program 6 — Count Vowels in String

```
def count_vowels(s):
    vowels = 'aeiouAEIOU'
    return sum(1 for ch in s if ch in vowels)

# Alternative
def count_vowels2(s):
    count = 0
    for ch in s.lower():
        if ch in 'aeiou':
            count += 1
    return count

print(count_vowels('Hello World'))  # 3
```

Program 7 — Find Duplicates in List

```
def find_duplicates(lst):
    seen = set()
    dups = set()
    for item in lst:
        if item in seen:
            dups.add(item)
        seen.add(item)
    return list(dups)

nums = [1,2,3,2,4,3,5]
print(find_duplicates(nums))   # [2, 3]
```

Program 8 — FizzBuzz (Classic)

```
def fizzbuzz(n):
    for i in range(1, n+1):
        if i % 15 == 0:        # divisible by both
            print('FizzBuzz')
        elif i % 3 == 0:
            print('Fizz')
        elif i % 5 == 0:
            print('Buzz')
        else:
            print(i)
```

```
fizzbuzz(15)
```

Program 9 — Count Word Frequency

```
def word_count(text):
    words = text.lower().split()
    freq  = {}
    for word in words:
        freq[word] = freq.get(word, 0) + 1
    return freq

text = 'the server is up the server is fine'
print(word_count(text))
# {'the':2,'server':2,'is':2,'up':1,'fine':1}

# Sort by frequency (most common first)
sorted_freq = sorted(freq.items(), key=lambda x: x[1], reverse=True)
```

Program 10 — Flatten Nested List

```
def flatten(lst):
    result = []
    for item in lst:
        if isinstance(item, list):  # if item is a list
            result.extend(flatten(item))  # recurse
        else:
            result.append(item)
    return result

nested = [1,[2,3],[4,[5,6]],7]
print(flatten(nested))   # [1,2,3,4,5,6,7]
```

Program 11 — Second Largest in List

```
def second_largest(lst):
    unique = list(set(lst))      # remove duplicates
    if len(unique) < 2:
        return None
    unique.sort(reverse=True)    # sort descending
    return unique[1]             # second item

print(second_largest([3,1,4,1,5,9,2]))  # 5
```

Program 12 — Anagram Check

```
def is_anagram(s1, s2):
    return sorted(s1.lower()) == sorted(s2.lower())

print(is_anagram('listen','silent'))  # True
print(is_anagram('hello','world'))    # False
```

Program 13 — SRE: Parse Log & Count Errors

```
# Sample log lines
```

```
logs = [
    'ERROR: disk 95% on web01',
    'INFO: cpu 45% on web02',
    'ERROR: mem 90% on db01',
    'WARN: latency high on cache01',
    'ERROR: connection timeout on web01',
]

def parse_logs(logs):
    counts = {'ERROR':0,'WARN':0,'INFO':0}
    errors = []

    for log in logs:
        level = log.split(':')[0]
        counts[level] = counts.get(level, 0) + 1
        if level == 'ERROR':
            errors.append(log)

    return counts, errors

counts, errors = parse_logs(logs)
print(counts)   # {'ERROR':3,'WARN':1,'INFO':1}
print(f'Total errors: {counts["ERROR"]}')
for e in errors:
    print(f'  !! {e}')
```

Program 14 — SRE: Find Down Servers

```
def find_down_servers(server_status):
    '''
    server_status: dict {hostname: True/False}
    Returns list of down servers
    '''
    return [host for host, up in server_status.items() if not up]

status = {
    'web01': True,
    'web02': False,
    'db01' : True,
    'cache': False,
}

down = find_down_servers(status)
print(f'Down servers: {down}')   # ['web02','cache']
```

Program 15 — Binary Search

```
def binary_search(lst, target):
    low, high = 0, len(lst) - 1

    while low <= high:
        mid = (low + high) // 2
        if lst[mid] == target:
            return mid         # found at index mid
        elif lst[mid] < target:
            low = mid + 1      # search right half
        else:
            high = mid - 1     # search left half

    return -1   # not found
```

```
lst = [2,5,8,12,16,23,38,56]
print(binary_search(lst, 23))   # 5  (index)
print(binary_search(lst, 10))   # -1 (not found)
```

Next → Chapter 10: Exception Handling | Chapter 11: OOP | Chapter 12: os/subprocess/re
Keep adding chapters as you learn — one notes file, always updated!

Chapter 9 — File I/O

open, read, write, append, with, os, json, csv

9.1 Opening Files — Modes

open(filepath, mode) — always use 'with' statement for safety.

Java	Python

`// Java try-with-resources`
`try(BufferedReader br`
`  =new BufferedReader(`
`  new FileReader("f.txt"))){`
`  String l=br.readLine();`
`} // auto closed`
	`# Python with statement`
	`with open('f.txt','r') as f:`
	`  content = f.read()`
	`# auto closed here`
	`# even if exception occurs`

Mode	Meaning	File exists?	File missing?
'r'	read only (default)	reads from start	FileNotFoundError
'w'	write — overwrites	wipes content	creates new file
'a'	append — add to end	adds at end	creates new file
'x'	create new only	FileExistsError	creates new file
'r+'	read + write	reads & writes	FileNotFoundError
'rb'	read binary	binary mode	FileNotFoundError
'wb'	write binary	binary mode	creates new file

> *'with open()' auto-closes file even if exception occurs. Like Java try-with-resources.*

9.2 Reading Files — 4 Ways

```
# 1. read() — entire file as one string
with open('server.log','r') as f:
    content = f.read()

# 2. readline() — one line at a time
with open('server.log','r') as f:
    line = f.readline()     # first line only

# 3. readlines() — all lines as list
with open('server.log','r') as f:
    lines = f.readlines()   # ['line1\n','line2\n',...]

# 4. Loop directly — BEST for large files
with open('server.log','r') as f:
    for line in f:          # reads one line at a time
        print(line.strip()) # strip removes \n
```

> *For large log files always loop directly: 'for line in f' — memory efficient, never loads full file.*

9.3 Writing & Appending Files

```
# Write — creates or OVERWRITES file
with open('report.txt','w') as f:
    f.write('Server Report\n')
    f.write('web01: UP\n')

# Write multiple lines
lines = ['web01: UP\n','db01: DOWN\n']
with open('report.txt','w') as f:
    f.writelines(lines)

# Append — ADDS to end, safe for logs
with open('report.txt','a') as f:
    f.write('cache01: UP\n')

# Write with f-string
hosts = ['web01','web02','db01']
with open('hosts.txt','w') as f:
    for host in hosts:
        f.write(f'{host}\n')

# Two files at once
with open('in.txt','r') as fin, open('out.txt','w') as fout:
    for line in fin:
        fout.write(line.upper())
```

Mode 'w' WIPES file first. Use 'a' to append. Easy to lose data — double check mode!

9.4 os Module — File & Directory Operations

```
import os

# —— Check existence ——
os.path.exists('/etc/hosts')           # True/False
os.path.isfile('/etc/hosts')           # True if file
os.path.isdir('/var/log')              # True if directory

# —— File info ——
os.path.getsize('server.log')          # bytes
os.path.basename('/var/log/app.log')   # 'app.log'
os.path.dirname('/var/log/app.log')    # '/var/log'
os.path.join('/var','log','app.log')   # '/var/log/app.log'

# —— Directory operations ——
os.listdir('/var/log')                 # list files
os.makedirs('/tmp/backup',exist_ok=True)  # create dir
os.rename('old.txt','new.txt')         # rename
os.remove('temp.txt')                  # delete file
os.getcwd()                            # current dir
os.chdir('/tmp')                       # change dir
os.environ.get('HOME')                 # env variable

# —— Walk directory tree ——
for root, dirs, files in os.walk('/var/log'):
    for file in files:
        full_path = os.path.join(root, file)
        print(full_path)
```

Always use os.path.join() to build paths — works on Linux and Windows.

9.5 Exception Handling with Files

```
try:
    with open('config.txt','r') as f:
        content = f.read()
except FileNotFoundError:
    print('File not found!')
except PermissionError:
    print('No read permission!')
except IOError as e:
    print(f'IO Error: {e}')
finally:
    print('Done — runs always')

# Check before opening
if os.path.exists('config.txt'):
    with open('config.txt','r') as f:
        data = f.read()
else:
    print('File missing')
```

> *Common exceptions: FileNotFoundError, PermissionError, IsADirectoryError, IOError*

9.6 JSON Files

JSON = standard format for configs and API responses.

Function	Direction	Use case
`json.load(f)`	file → dict	read JSON config file
`json.dump(d,f)`	dict → file	save config to JSON
`json.loads(str)`	string → dict	parse API response
`json.dumps(d)`	dict → string	convert for sending

```
import json

# Read JSON file
with open('config.json','r') as f:
    config = json.load(f)          # file → dict
print(config.get('host','N/A'))

# Write JSON file
server = {'host':'web01','port':8080,'active':True}
with open('server.json','w') as f:
    json.dump(server, f, indent=4) # dict → file

# Parse JSON string (API response)
response = '{"status":"ok","code":200}'
data = json.loads(response)        # string → dict
print(data['status'])              # ok

# Dict to JSON string
s = json.dumps(server, indent=4)   # dict → string
```

> *Memory trick: 's' suffix = string version. load=file, loads=string. dump=file, dumps=string.*

9.7 CSV Files

```
import csv
```

```
# Read CSV — rows as lists
with open('servers.csv','r') as f:
    reader = csv.reader(f)
    header = next(reader)          # skip header
    for row in reader:
        print(row)                 # ['web01','UP','80']

# Read CSV — rows as dicts (header as keys)
with open('servers.csv','r') as f:
    for row in csv.DictReader(f):
        print(row['host'], row['status'])

# Write CSV
data = [['host','status'],['web01','UP'],['db01','DOWN']]
with open('out.csv','w',newline='') as f:
    csv.writer(f).writerows(data)

# Write CSV from dicts
rows = [{'host':'web01','cpu':85},{'host':'db01','cpu':45}]
with open('report.csv','w',newline='') as f:
    w = csv.DictWriter(f, fieldnames=['host','cpu'])
    w.writeheader()
    w.writerows(rows)
```

Always use newline='' when writing CSV to avoid extra blank lines on Windows.

9.8 Chapter 9 — Quick Cheatsheet

Task	Code
Read whole file	`with open('f','r') as f: content=f.read()`
Read line by line	`for line in f: print(line.strip())`
Read into list	`lines=f.readlines()`
Write file	`with open('f','w') as f: f.write('text')`
Append to file	`with open('f','a') as f: f.write('more')`
Two files at once	`with open(a,'r') as fi, open(b,'w') as fo:`
File exists?	`os.path.exists('f.txt')`
Is file / dir?	`os.path.isfile()  /  os.path.isdir()`
File size	`os.path.getsize('f.txt')`
Build path	`os.path.join('/var','log','app.log')`
List directory	`os.listdir('/var/log')`
Create directory	`os.makedirs('/tmp/bk',exist_ok=True)`
Delete file	`os.remove('file.txt')`
Walk dir tree	`for root,dirs,files in os.walk('/var'):`
Read JSON file	`json.load(f)  →  dict`
Write JSON file	`json.dump(d,f,indent=4)`
Parse JSON string	`json.loads(api_response)`
Dict to JSON string	`json.dumps(d)`
Read CSV as list	`csv.reader(f)`
Read CSV as dict	`csv.DictReader(f)`
Write CSV	`csv.writer(f).writerows(data)`
File error handling	`except FileNotFoundError / PermissionError`

9.9 Real SRE Examples

Log Parser — Count Errors

```
import os

def parse_log(filepath):
    if not os.path.exists(filepath):
        print(f'File not found: {filepath}')
        return
    counts={'ERROR':0,'WARN':0,'INFO':0}
    errors=[]
    with open(filepath,'r') as f:
        for line in f:
            line=line.strip()
            if not line: continue
            for level in counts:
                if line.startswith(level):
                    counts[level]+=1
                    if level=='ERROR': errors.append(line)
    print(f'Summary: {counts}')
    for e in errors: print(f'  >> {e}')
```

Save Health Report to JSON

```
import json
from datetime import datetime

def save_health(hosts, outfile='health.json'):
    report={'timestamp':datetime.now().strftime('%Y-%m-%d %H:%M:%S'),
            'servers':[{'host':h,'status':'UP'} for h in hosts]}
    with open(outfile,'w') as f:
        json.dump(report,f,indent=4)
    print(f'Saved to {outfile}')

save_health(['web01','web02','db01'])
```

Backup .conf Files

```
import os, shutil
from datetime import datetime

def backup_configs(src,dst):
    ts=datetime.now().strftime('%Y%m%d_%H%M%S')
    out=os.path.join(dst,f'backup_{ts}')
    os.makedirs(out,exist_ok=True)
    count=0
    for f in os.listdir(src):
        if f.endswith('.conf'):
            shutil.copy2(os.path.join(src,f),os.path.join(out,f))
            count+=1
    print(f'{count} files backed up to {out}')

backup_configs('/etc/nginx','/tmp/backups')
```

Interview Questions & Answers — All Chapters

Ch 1–8 Quick Answers

```
# mutable vs immutable?
# Mutable: list,dict,set   Immutable: int,float,str,tuple,bool

# is vs ==?  is=same object  ==  =same value. Use is for None only.
# input() always returns str — port=int(input('Port: '))
# sort() vs sorted()?  sort()=in-place  sorted()=new list
# *args=tuple  **kwargs=dict  order: def f(a,*args,**kwargs)
# mutable default bug: use lst=None then if lst is None: lst=[]
# recursion base case: if n==0: return 1  — stops infinite call
# for/else: else runs only if loop had NO break
# lambda: sq=lambda x:x*x  used with sorted/filter/map
# append vs extend: append adds ONE item  extend merges list
# shallow vs deep copy: copy.copy() vs copy.deepcopy()
# GIL: only one thread runs at a time in CPython
```

Ch 9 — File I/O Questions

Q1. Why use 'with open()' instead of open()?

```
# 'with' is a context manager
# Closes file automatically — even if exception occurs
# Same as Java try-with-resources
with open('f.txt','r') as f:   # ✅always safe
    data=f.read()
```

Q2. read() vs readline() vs readlines()?

```
f.read()       # entire file → one string
f.readline()   # one line   → string
f.readlines()  # all lines  → list
for line in f: # one line at a time → most memory efficient
```

Q3. Difference between 'w' and 'a' mode?

```
'w'  # WRITE — wipes existing content first!
'a'  # APPEND — adds to end, existing content safe
# SRE rule: 'a' for logs, 'w' for fresh reports
```

Q4. json.load() vs json.loads()?

```
json.load(f)       # FILE   → dict
json.loads(string) # STRING → dict
json.dump(d, f)    # dict   → FILE
json.dumps(d)      # dict   → STRING
# 's' suffix always means string version
```

Q5. How to read large file without loading into memory?

```
# Loop directly — reads one line at a time
with open('huge.log','r') as f:
    for line in f:
        process(line)   # never loads full file

# AVOID for large files:
```

```
content=f.read()        # loads entire file into memory
lines=f.readlines()     # loads entire file into memory
```

File I/O Programs

Program: Count lines, words, chars in file

```
def file_stats(path):
    lines=words=chars=0
    with open(path,'r') as f:
        for line in f:
            lines+=1
            words+=len(line.split())
            chars+=len(line)
    print(f'Lines:{lines} Words:{words} Chars:{chars}')
```

Program: Search keyword in file (grep-like)

```
def grep(path, keyword):
    with open(path,'r') as f:
        for i,line in enumerate(f,1):
            if keyword.lower() in line.lower():
                print(f'Line {i}: {line.strip()}')

grep('app.log','ERROR')
```

Program: Read JSON config safely

```
import json

def load_config(path):
    try:
        with open(path,'r') as f:
            return json.load(f)
    except FileNotFoundError:
        print('Config missing!') ; return {}
    except json.JSONDecodeError:
        print('Bad JSON!') ; return {}

config=load_config('server.json')
host=config.get('host','localhost')
```

Program: Write server report to CSV

```
import csv

def write_report(servers, outfile='report.csv'):
    with open(outfile,'w',newline='') as f:
        w=csv.DictWriter(f,fieldnames=['host','status','cpu'])
        w.writeheader()
        w.writerows(servers)
    print(f'Report saved: {outfile}')

write_report([
    {'host':'web01','status':'UP','cpu':85},
    {'host':'db01', 'status':'DOWN','cpu':10},
])
```

Next → Chapter 10: Exception Handling | Chapter 11: OOP | Chapter 12: os/subprocess/re
Keep adding chapters as you learn — one notes file, always updated!